

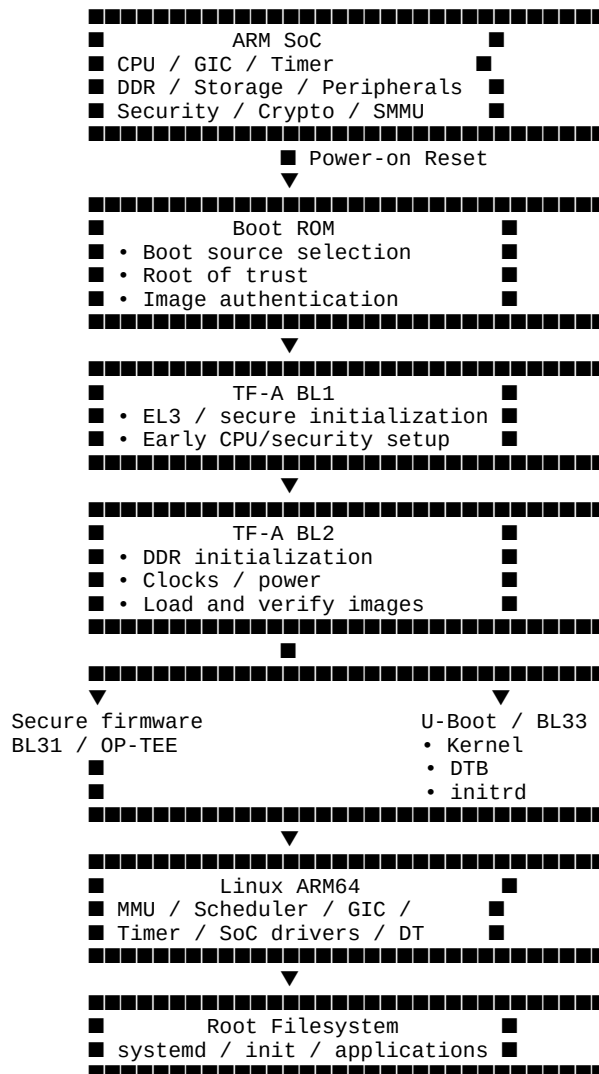
ARM SoC BSP & Boot Architecture

High-Level Design (HLD) + Low-Level Design (LLD) + C++ Reference Implementation

Interview focus: Senior SDE / Kernel Engineer — ARM firmware, BSP, bootloader, Linux bring-up, secure boot, recovery, and production architecture.

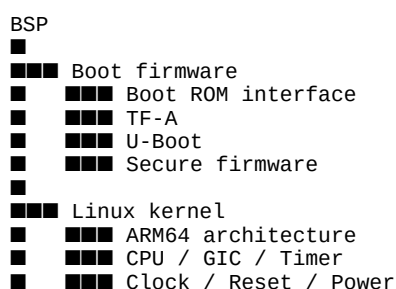
Part I — High-Level Design (HLD)

1. Overall boot architecture



2. BSP definition

For a new ARM SoC, the BSP is the complete software package required to bring a specific SoC/board from reset to a usable operating system. It is broader than just peripheral drivers.



- ■■■ GPIO / UART / I2C / SPI
- ■■■ DMA / SMMU
- ■■■ PCIe / USB / Ethernet
- ■■■ Storage
-
- Device Tree
- ■■■ SoC .dtsi
- ■■■ Board .dts
-
- Root filesystem
- ■■■ init / systemd / applications
-
- Build / deployment
- Yocto / Buildroot
- image generation
- signing
- OTA / A-B update

3. Boot stages

Reset: CPU starts from the reset vector. MMU, DRAM and most platform services are not yet available.

Boot ROM: Immutable silicon code selects the boot source, performs minimal initialization, authenticates the first image, and transfers control.

BL1/BL2: TF-A stages establish secure execution, initialize DDR and clocks, authenticate/load later stages, and prepare the platform.

BL31: Provides EL3 runtime services such as PSCI and secure monitor functionality.

U-Boot: Loads the kernel, Device Tree and initrd, establishes boot arguments, and transfers control to Linux.

Linux: Initializes MMU, memory management, scheduler, interrupts, timers, drivers and userspace.

4. DDR bring-up

DDR Controller

-
- Controller configuration
- PHY configuration
- Timing configuration
- Training
 - ■■■ Write leveling
 - ■■■ Read leveling
 - ■■■ DQ/DQS training
 - ■■■ Vref / timing calibration
- Address mapping
- ECC configuration

DDR is one of the most SoC- and board-specific parts of the BSP. Reliable boot normally depends on successfully completing DDR initialization before loading larger firmware images.

5. Device Tree

arch/arm64/boot/dts/vendor/
 ■■■ soc-x.dtsi
 ■■■ board-a.dts
 ■■■ board-b.dts
 ■■■ board-c.dts

The SoC .dtsi describes common hardware such as CPU, GIC, timer, UART, I2C, SPI, DMA and PCIe. Board .dts files add board-specific regulators, GPIO assignments, PMICs, storage and other physical configuration.

6. Interrupt, clock, reset and power architecture

Peripheral → Linux subsystem → SoC driver → framework → hardware

UART / I2C / SPI / DMA / PCIe / Ethernet

```

  ■
  ▼
GIC
  ■
  ▼
CPU

```

Use standard Linux frameworks for clocks, resets, regulators, power domains, interrupts, DMA, networking, storage and USB rather than embedding cross-subsystem register manipulation inside each peripheral driver.

7. DMA and SMMU

Without SMMU:

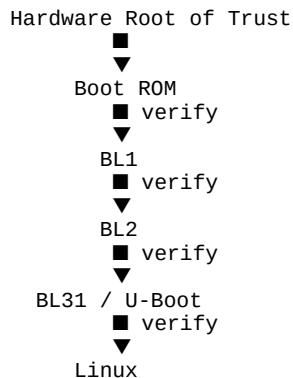
Device ■■ DMA ■■> Physical memory

With SMMU:

Device ■■ DMA address ■■> SMMU ■■ translation/protection ■■> Memory

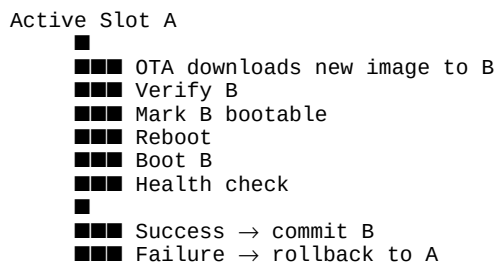
A modern ARM SoC should consider SMMU early for DMA isolation, virtualization, address translation and security.

8. Secure boot



Production security can include eFuse/OTP root-key hashes, signed images, rollback protection, measured boot, TrustZone and secure key storage.

9. A/B update and recovery



Watchdogs, boot-attempt counters, recovery mode and atomic A/B updates prevent failed firmware updates from permanently bricking devices.

10. BSP bring-up phases

Phase 1: CPU → UART → SRAM → DDR

Phase 2: ROM → BL1 → BL2 → BL31 → U-Boot

Phase 3: ARM64 Linux minimum: CPU / GIC / Timer / MMU / UART

Phase 4: Storage / Ethernet / USB / I2C / SPI / GPIO / PCIe

Phase 5: Performance: DVFS / DDR / DMA / interrupt affinity / SMP

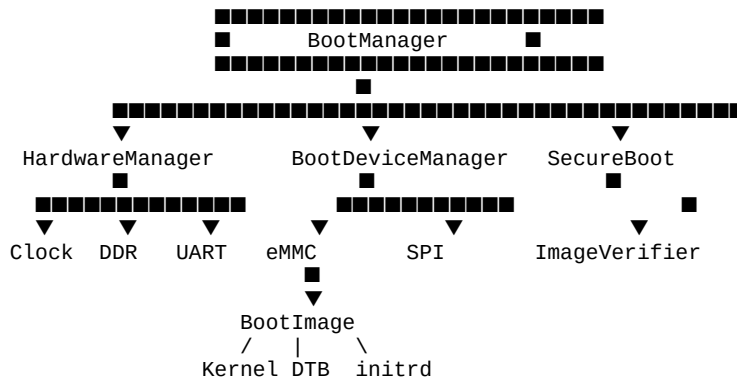
Phase 6: Production: secure boot / OTA / A-B / watchdog / crash dump / telemetry

Part II — Low-Level Design (LLD)

11. LLD scope

The LLD focuses on a Boot Manager responsible for hardware initialization, boot-device selection, image loading, secure image verification, kernel handoff and recovery.

12. LLD component diagram



13. Main classes and interfaces

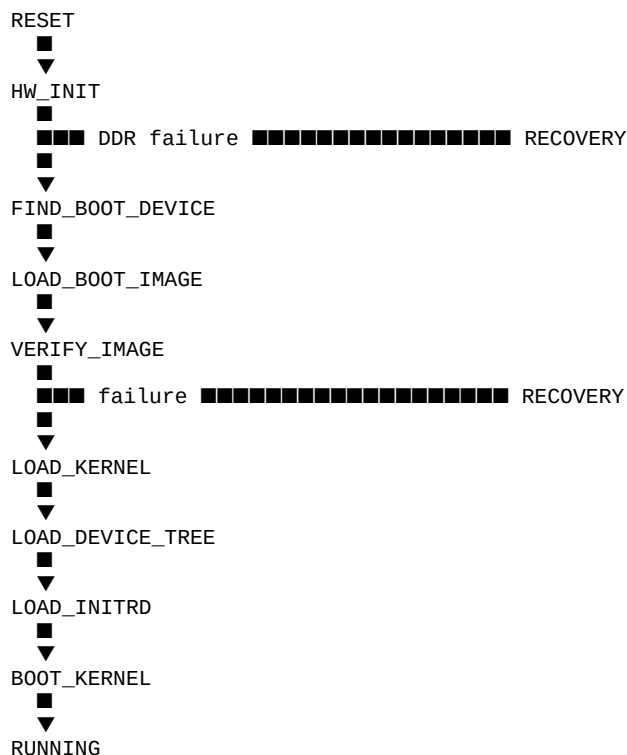
BootManager

- IHardwareController
- IBootDevice
- IImageVerifier
- IImageLoader
- IRecoveryManager

Implementations:

- ARMHardwareController
- EMMCDevice / SPINORDevice
- SecureImageVerifier
- BootImageLoader
- RecoveryManager

14. Boot state machine



15. C++ interfaces

```
enum class BootDeviceType
{
    EMMC,
    SPI_NOR,
    USB,
    NETWORK
};

class IBootDevice
{
public:
    virtual ~IBootDevice() = default;

    virtual bool probe() = 0;

    virtual bool read(
        uint64_t offset,
        void* buffer,
        size_t size) = 0;

    virtual BootDeviceType getType() const = 0;
};

class IHardwareController
{
public:
    virtual ~IHardwareController() = default;

    virtual bool initializeClocks() = 0;
    virtual bool initializeDDR() = 0;
    virtual bool initializeSecurity() = 0;

    virtual void resetSystem() = 0;
};

class IImageVerifier
{
public:
    virtual ~IImageVerifier() = default;

    virtual bool verify(
        const void* image,
        size_t size) = 0;
};

class IImageLoader
{
public:
    virtual ~IImageLoader() = default;

    virtual bool loadKernel(
        IBootDevice& device,
        BootImage& image) = 0;

    virtual bool loadDTB(
        IBootDevice& device,
        BootImage& image) = 0;

    virtual bool loadInitrd(
        IBootDevice& device,
        BootImage& image) = 0;
};

class IRecoveryManager
{
public:
    virtual ~IRecoveryManager() = default;

    virtual bool shouldEnterRecovery() = 0;
    virtual bool enterRecovery() = 0;
    virtual void markBootSuccess() = 0;
    virtual void markBootFailure() = 0;
};
```

16. Boot image model

```
struct BootImage
{
    void* kernelAddress;
    size_t kernelSize;

    void* dtbAddress;
```

```

        size_t dtbSize;

        void* initrdAddress;
        size_t initrdSize;

        uint64_t entryPoint;
    };

```

17. eMMC and SPI-NOR implementations

```

class EMMCDevice : public IBootDevice
{
public:
    bool probe() override
    {
        // Initialize eMMC controller.
        // Send identification commands.
        // Check card state.
        return true;
    }

    bool read(
        uint64_t offset,
        void* buffer,
        size_t size) override
    {
        // Program DMA.
        // Read blocks from eMMC.
        // Validate transfer.
        return true;
    }

    BootDeviceType getType() const override
    {
        return BootDeviceType::EMMC;
    }
};

class SPINORDevice : public IBootDevice
{
public:
    bool probe() override;
    bool read(
        uint64_t offset,
        void* buffer,
        size_t size) override;

    BootDeviceType getType() const override
    {
        return BootDeviceType::SPI_NOR;
    }
};

```

18. ARM hardware controller

```

class ARMHardwareController : public IHardwareController
{
public:
    bool initializeClocks() override
    {
        configurePLL();
        configureBusClock();
        configurePeripheralClock();
        return true;
    }

    bool initializeDDR() override
    {
        configureDDRController();
        configureDDRPHY();

        if (!runDDRTraining())
            return false;

        return true;
    }

    bool initializeSecurity() override
    {
        configureTrustZone();
        configureSecureRegions();
        return true;
    }
}

```

```

    void resetSystem() override
    {
        writeRegister(SYSTEM_RESET_REGISTER, RESET_VALUE);
    }

private:
    void configurePLL();
    void configureBusClock();
    void configurePeripheralClock();
    void configureDDRController();
    void configureDDRPHY();
    bool runDDRTraining();
    void configureTrustZone();
    void configureSecureRegions();

    void writeRegister(uintptr_t address, uint32_t value);
};

```

19. Secure image verifier

```

class SecureImageVerifier : public IImageVerifier
{
public:
    bool verify(
        const void* image,
        size_t size) override
    {
        if (!validateSignature(image, size))
            return false;

        if (!checkRollbackCounter(image))
            return false;

        if (!checkHash(image, size))
            return false;

        return true;
    }

private:
    bool validateSignature(
        const void* image,
        size_t size);

    bool checkRollbackCounter(
        const void* image);

    bool checkHash(
        const void* image,
        size_t size);
};

```

20. Recovery manager interface

```

class IRecoveryManager
{
public:
    virtual ~IRecoveryManager() = default;

    virtual bool shouldEnterRecovery() = 0;
    virtual bool enterRecovery() = 0;
    virtual void markBootSuccess() = 0;
    virtual void markBootFailure() = 0;
};

```

21. BootManager — central orchestration class

```
class BootManager
{
public:
    BootManager(
        IHardwareController& hardware,
        std::vector<IBootDevice*> devices,
        IImageVerifier& verifier,
        IImageLoader& loader,
        IRecoveryManager& recovery)
        : hardware_(hardware),
          devices_(std::move(devices)),
          verifier_(verifier),
          loader_(loader),
          recovery_(recovery)
    {
    }

    bool boot()
    {
        if (recovery_.shouldEnterRecovery())
            return recovery_.enterRecovery();

        if (!initializeHardware())
            return handleBootFailure();

        IBootDevice* device = findBootDevice();

        if (device == nullptr)
            return handleBootFailure();

        BootImage image{};

        if (!loadImage(*device, image))
            return handleBootFailure();

        if (!verifyImage(image))
            return handleBootFailure();

        if (!transferControl(image))
            return handleBootFailure();

        recovery_.markBootSuccess();

        return true;
    }

private:
    bool initializeHardware()
    {
        if (!hardware_.initializeClocks())
            return false;

        if (!hardware_.initializeDDR())
            return false;

        if (!hardware_.initializeSecurity())
            return false;

        return true;
    }

    IBootDevice* findBootDevice()
    {
        for (auto* device : devices_)
        {
            if (device->probe())
                return device;
        }

        return nullptr;
    }

    bool loadImage(
        IBootDevice& device,
        BootImage& image)
    {
        if (!loader_.loadKernel(device, image))
            return false;

        if (!loader_.loadDTB(device, image))
            return false;

        if (!loader_.loadInitrd(device, image))
```



```

        return false;
    }
    return true;
}

bool verifyImage(const BootImage& image)
{
    if (!verifier_.verify(
        image.kernelAddress,
        image.kernelSize))
        return false;

    if (!verifier_.verify(
        image.dtbAddress,
        image.dtbSize))
        return false;

    if (!verifier_.verify(
        image.initrdAddress,
        image.initrdSize))
        return false;

    return true;
}

bool transferControl(const BootImage& image)
{
    // ARM64 Linux boot protocol:
    // x0 = DTB address
    // x1-x3 = reserved
    // branch to kernel entry point.

    jumpToKernel(
        image.entryPoint,
        reinterpret_cast<uint64_t>(
            image.dtbAddress));

    return true;
}

bool handleBootFailure()
{
    recovery_.markBootFailure();
    return recovery_.enterRecovery();
}

void jumpToKernel(
    uint64_t entryPoint,
    uint64_t dtbAddress)
{
    // Architecture-specific assembly.
}

private:
    IHardwareController& hardware_;
    std::vector<IBootDevice*> devices_;
    IImageVerifier& verifier_;
    IImageLoader& loader_;
    IRecoveryManager& recovery_;
};

```

22. Explicit error model

```

enum class BootError
{
    NONE,
    CLOCK_INIT_FAILED,
    DDR_INIT_FAILED,
    SECURITY_INIT_FAILED,
    NO_BOOT_DEVICE,
    IMAGE_READ_FAILED,
    KERNEL_VERIFY_FAILED,
    DTB_VERIFY_FAILED,
    INITRD_VERIFY_FAILED,
    ROLLBACK_DETECTED,
    KERNEL_BOOT_FAILED
};

struct BootResult
{
    bool success;
    BootError error;
};

```

23. Unit-testability with mocks

```
class MockBootDevice : public IBootDevice
{
public:
    bool probe() override
    {
        return true;
    }

    bool read(
        uint64_t,
        void*,
        size_t) override
    {
        return true;
    }

    BootDeviceType getType() const override
    {
        return BootDeviceType::EMMC;
    }
};
```

The same BootManager can be tested with MockHardware, MockBootDevice, MockVerifier, MockImageLoader and MockRecovery. No real ARM hardware is required for orchestration-level tests.

24. Important firmware/C++ distinction

The class design above represents the logical LLD and dependency boundaries. The earliest boot stages are not ordinary C++ applications: reset-vector code and very early initialization normally use assembly and C because a stack, DRAM, allocator and full C++ runtime are not yet available. C++ can be used selectively in later firmware/bootloader components where the runtime environment is established.

25. Senior-level interview talking points

1. Dependency inversion:
BootManager depends on interfaces, not eMMC/SPI implementations.
2. Hardware abstraction:
Register-level access is isolated behind hardware-controller classes.
3. Security:
Image authentication and rollback checks occur before kernel handoff.
4. Recovery:
Boot failures update persistent state and can trigger recovery/A-B rollback.
5. Testability:
Hardware dependencies can be replaced by mocks.
6. Linux integration:
Device Tree describes hardware while Linux subsystems own common functionality.
7. Production:
Watchdog, crash logs, A/B updates and telemetry are part of the architecture.
8. Real firmware constraint:
Reset/early-init code is assembly/C; C++ is used only where runtime support exists.

26. Suggested next LLD deep-dive

For a stronger Kernel Engineer interview exercise, the next subsystem to design in LLD is DDR initialization. A useful decomposition is DDRManager → DDRController → DDRPHY → DDRTraining → DDRTimingConfig → DDRCalibration → DDRHealthMonitor, followed by register-level C/C++ code, interrupt/callback handling, state transitions and failure recovery.